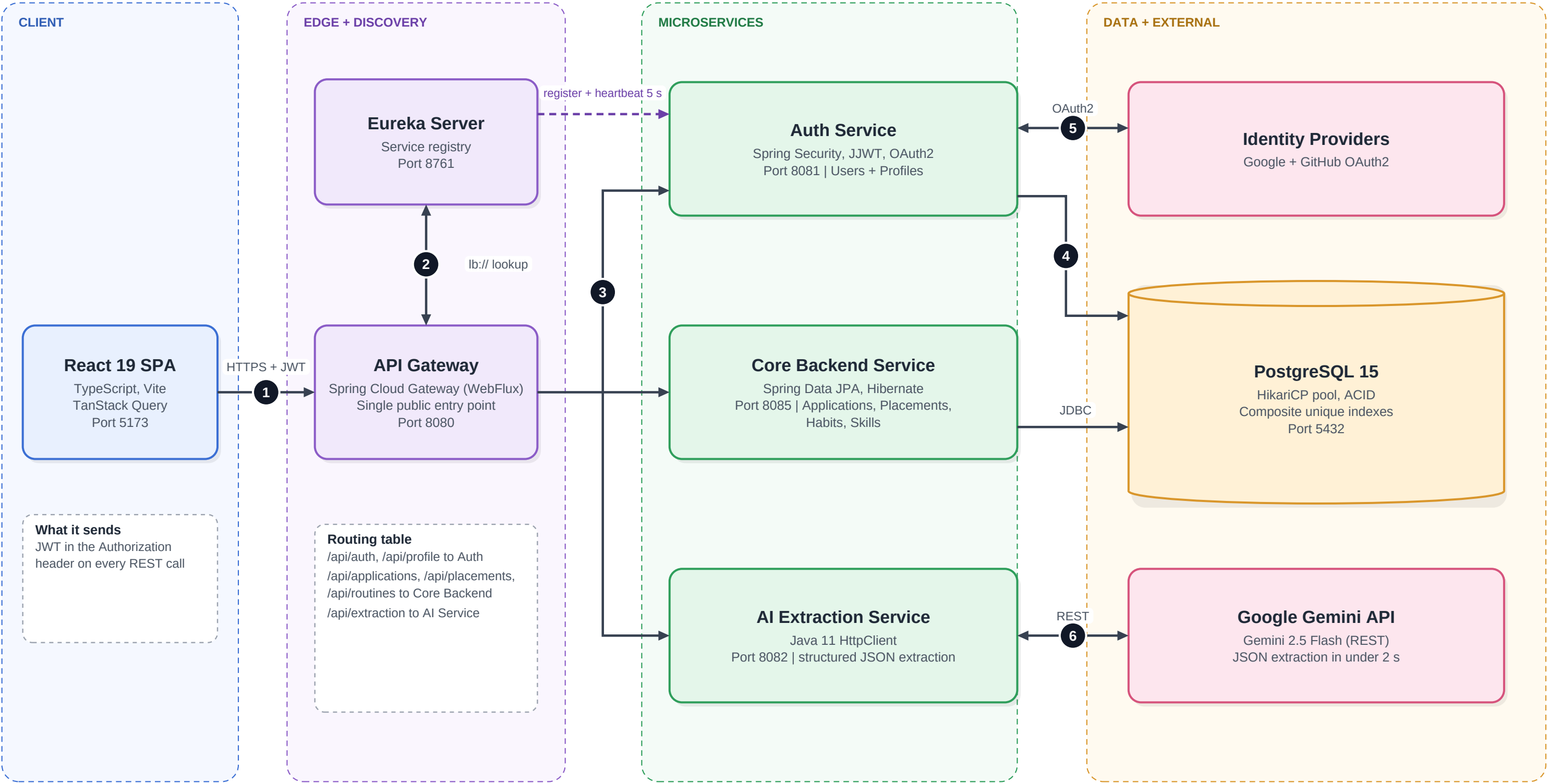


Career OS

Distributed cloud-native microservices | React 19, Spring Boot 3.3, Spring Cloud Gateway, Eureka, PostgreSQL 15, Gemini 2.5 Flash



REQUEST FLOW (follow the numbers)

→ synchronous call

- - - - -> async / background

- React SPA calls REST APIs over HTTPS with a JWT.
- Gateway asks Eureka for a live instance (lb://service-name).
- Gateway routes by URL path to Auth, Core Backend or AI service.

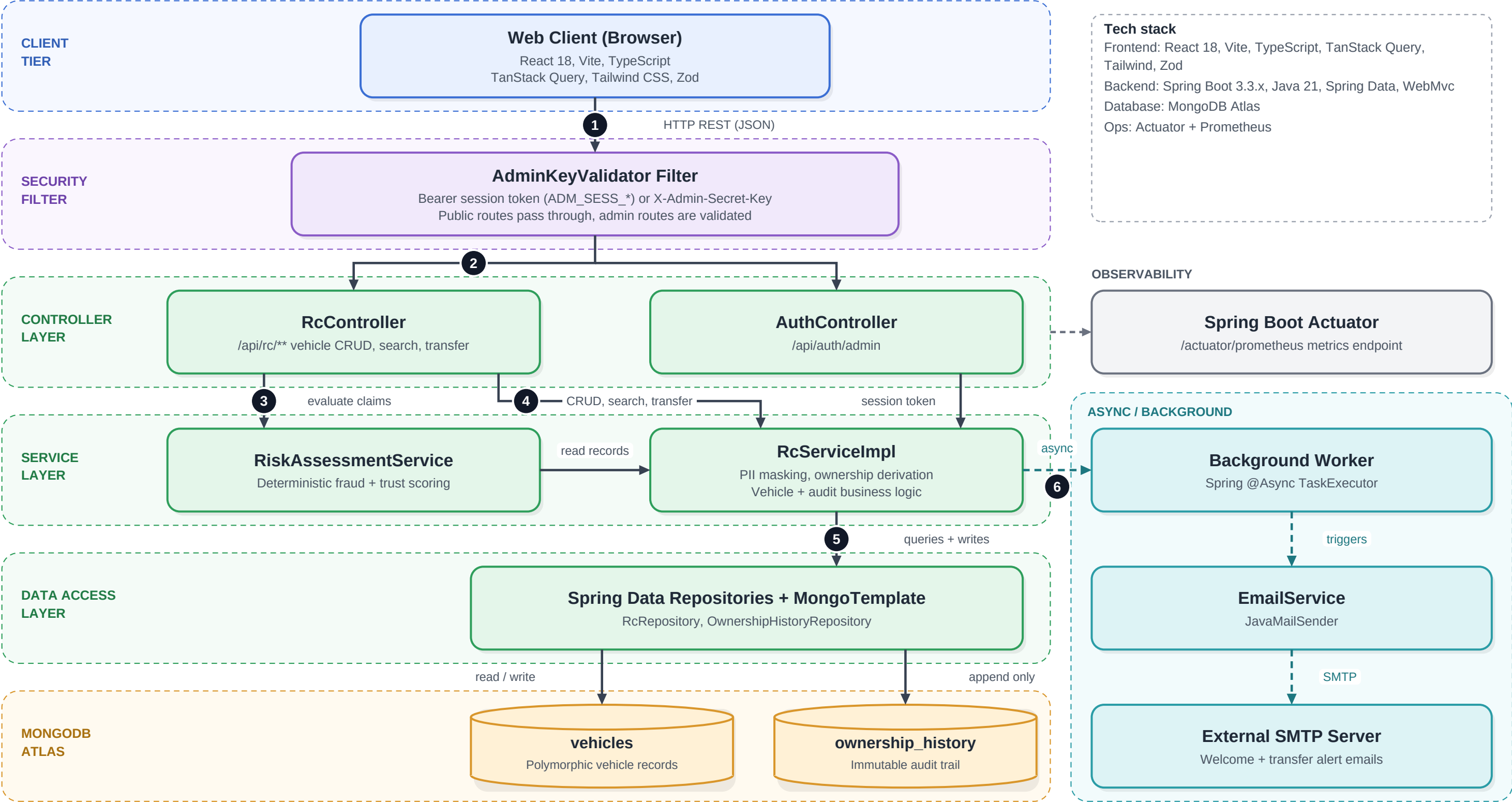
- Auth + Core Backend read and write PostgreSQL over JDBC.
- Auth Service completes the Google / GitHub OAuth2 handshake.
- AI Service calls Gemini 2.5 Flash and returns structured JSON.

KEY DESIGN DECISIONS

- Only the Gateway is public. Services stay internal behind it.
- Eureka gives discovery + load balancing, so instances scale or fail over with no config change.
- Auth owns Users + Profiles. Core Backend owns Applications, Placements, Habits, Skills.
- AI extraction is its own service, so slow LLM calls never block core CRUD.

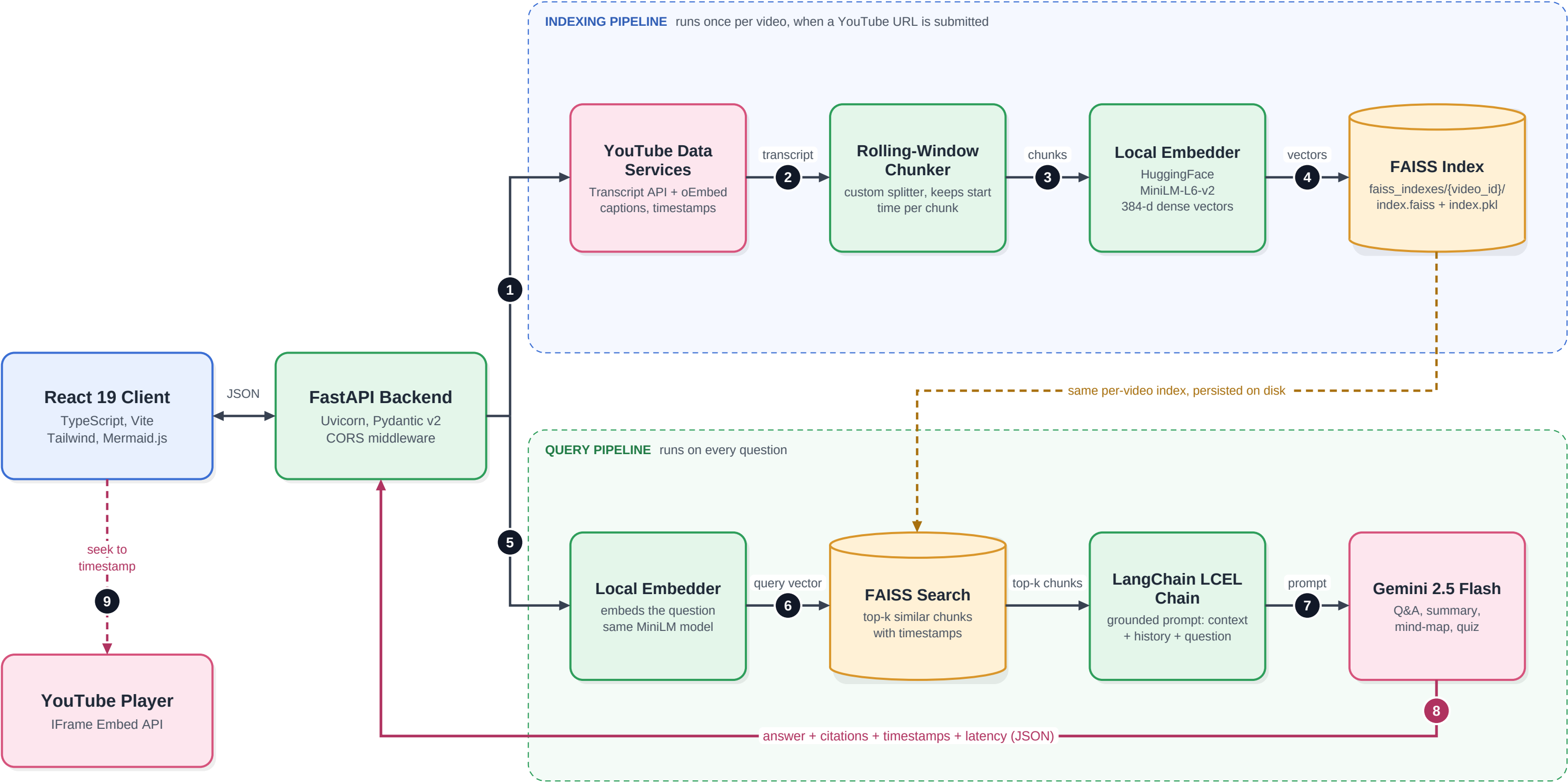
Drive Verify

Vehicle registration and fraud verification | React 18, Spring Boot 3.3, Java 21, MongoDB Atlas, Prometheus



Video-Mind AI (YT_ChatBot)

RAG over YouTube videos | React 19, FastAPI, LangChain LCEL, MiniLM embeddings, FAISS, Gemini 2.5 Flash



REQUEST FLOW (follow the numbers)

→ synchronous call

---> async / background

- 1 User pastes a YouTube URL. React posts it to FastAPI (validated).
- 2 Backend fetches captions + timestamps (YouTube Transcript API).
- 3 Rolling-window chunker splits the transcript, keeps start times.
- 4 MiniLM embeds chunks locally. Vectors saved per video_id.
- 5 User asks a question. FastAPI passes it + chat history to LCEL.
- 6 Question is embedded. FAISS returns top-k chunks + timestamps.
- 7 LCEL builds a grounded prompt and calls Gemini 2.5 Flash.
- 8 Answer + citations return as JSON. (9) A citation click seeks the player.

KEY DESIGN DECISIONS

- Two pipelines: index once, then query many times.
- Local MiniLM embeddings: no per-call embedding API cost.
- Chunks keep timestamps, so citations are grounded and clickable.
- One FAISS index per video_id keeps videos isolated.

How to Present an Architecture Diagram in Interviews

What interviewers look for, and a 60-second talk track you can reuse for any project

THE 6 RULES

1

Flow left to right, top to bottom

User on the left, data on the right. Re-route lines until arrows stop crossing.

2

Keep it to 5 to 8 boxes

Group them into layers: Client, Edge, Services, Data, External. Merge small pieces.

3

Name boxes by responsibility

Bold name on top, one small line of tech below. Leave out ports, versions and library lists.

4

Number the arrows

Number them in the order a real request travels, then narrate: "one, two, three".

5

Solid = sync, dashed = async

Say the legend out loud once. Only draw arrows that carry real data flow.

6

End with trade-offs

Give 2 or 3 decisions and why: gateway, async email, local FAISS. Reasoning scores more than boxes.

Whiteboard tip: rectangle = service, cylinder = database, cloud = external API.
Draw the happy path first. Add async and failure paths only if asked.

60-SECOND TALK TRACK

0:00

One-line purpose

"This is [system]. It lets users [do X]."

0:10

Walk the numbered flow

"A request starts at [client], hits [entry point], goes to [service], then reads [store]."

0:40

Explain the data

"Data lives in [DB] because [reason: ACID, flexible schema, low latency]."

0:50

Trade-off + scale

"I chose [A] over [B]. At 10x scale I would add [cache, queue, replicas]."

COMMON MISTAKES

- Putting the full tech stack, ports and paths inside every box.
- Auto-generated Mermaid layouts with tangled, crossing arrows.
- Drawing 15+ boxes, so you spend the interview reading, not explaining.
- No numbering, no legend, no order of events.
- Skipping the "why": a diagram without decisions sounds like a list.